

리뷰 기반 챗봇 프로토타입 만들기

이 유인물만 보고 따라 하면 브라우저에서 바로 돌아가는 리뷰 챗봇 프로토타입을 만들 수 있음. 서버 연결, API 키 발급, 배포 같은 건 다 뺐음. HTML 파일 하나 만들어서 크롬으로 열면 끝.

0. 뭘 만드는 건지 먼저 알기

사용자가 "스터디카페 시험기간에도 조용해?" 같은 질문을 입력하면, 미리 준비해둔 리뷰 데이터에서 답을 찾아 보여주는 챗봇.

- 실시간으로 AI가 답을 생성하는 게 아니라, **질문 - 답변을 미리 다 짚지어놓고**, 사용자가 입력한 문장이 어떤 질문이랑 제일 비슷한지 찾아서 그 답을 보여주는 방식.
- 그래서 진짜 LLM API를 연결할 필요가 없음. 완성 코드를 HTML 파일 하나로 저장해서 브라우저에서 열기만 하면 됨.

화면 상태는 6개로 나뉨 (이거 기준으로 UI랑 기능을 같이 설계할 거임)

상태	언제 뜨는지
대기	질문 입력 전
로딩	질문 보낸 직후
확실 답변	리뷰 의견이 한쪽으로 쏠릴 때
의견 갈림 답변	긍정/부정 비율이 7:3 안쪽일 때
정보부족	관련 리뷰가 너무 적을 때
이해실패	질문이랑 매칭되는 카테고리가 없을 때

1. 사용할 툴

툴	용도
네이버 리뷰 / 블로그, 카카오맵	리뷰 원본 데이터 수집
크롬 개발자도구 (F12)	리뷰 텍스트만 뽑아내기 (크롤링)
Gemini / ChatGPT	리뷰 표 정리 + 라벨링 교차검증 + 최종 코드 생성
구글 스프레드시트 / 엑셀	라벨링 결과 정리, 질문-답변 데이터 정리
메모장(또는 텍스트 편집기)	완성 코드를 .html 파일로 저장
크롬 브라우저	완성된 .html 파일 실행
Framer AI	화면 디자인 시안 참고용 (실제 기능 연결은 안 함)

2. 전체 구조 한눈에 보기

```
리뷰 원본 수집 (크롤링)
↓
리뷰 표 정리 + 라벨링 (GPT/Claude/Gemini 교차검증 → 카테고리 + 톤 확정)
↓
질문-답변 데이터셋 구성
↓
화면 디자인 (Framer AI 참고용 시안)
↓
디자인 시안 → HTML/CSS 코드로 변환
↓
데이터 기반 작동 로직(JS) 추가
↓
메모장에 붙여넣고 .html로 저장
↓
크롬으로 열어서 실행
```

3. Step 1 — 레퍼런스 조사하기

무작정 시작하지 말고, 비슷한 서비스 3~5개 먼저 캡처해두기.

- 검색어 예시: "리뷰 요약 챗봇", "리뷰 기반 Q&A", "review sentiment app UI"
- 볼 포인트
 - 질문을 어떻게 입력받는지 (텍스트박스 vs 추천 질문 칩)
 - 답변을 어떤 형태로 보여주는지 (한 줄 요약 vs 카드형)
 - 신뢰도나 근거를 어떻게 표시하는지 (별점, 퍼센트, 리뷰 개수)
- 캡처는 피그잼이나 파워포인트 한 장에 모아놓기. 나중에 화면 디자인 참고 자료로 씬.

4. Step 2 — 리뷰 원본 데이터 수집하기 (크롤링)

네이버 리뷰, 네이버 블로그, 카카오맵 리뷰에서 리뷰 텍스트만 뽑아내는 방법.

4-1. 개발자도구로 리뷰 요소 찾기

1. 리뷰 보고 싶은 페이지(네이버 플레이스, 카카오맵 등) 접속
2. 키보드 **F12** 누르기 (또는 우클릭 → 검사) → 개발자도구 열림
3. 개발자도구 좌측 상단에 있는 **화살표+박스 아이콘**(요소 선택 도구) 클릭
4. 화면에서 리뷰 텍스트 위에 마우스 올리기 → 해당 부분이 파란색으로 하이라이트되면서 개발자도구 오른쪽(Elements 탭)에 그 부분의 HTML 코드가 바로 뜸
5. 리뷰 텍스트를 감싸고 있는 태그와 **class 이름**을 확인 (예: `class="review_text"` 같은 식)

리뷰마다 class 이름이 똑같이 반복되는 걸 찾는 게 포인트. 그래야 다음 단계에서 한 번에 다 뽑을 수 있음.

4-2. 콘솔에서 한 번에 텍스트 뽑기 (제일 중요한 부분)

리뷰를 하나씩 드래그해서 복사하면 100개 모으는 데 한참 걸림. **콘솔 명령어 한 줄이면 그 페이지에 있는 리뷰를 전부 한꺼번에 뽑을 수 있음** — 이게 크롤링에서 시간을 제일 아껴주는 부분이라 꼭 익혀두기.

1. 개발자도구 상단 탭에서 **Console** 탭 클릭
2. 아래처럼 코드 입력 후 Enter (class 이름은 4-1에서 확인한 걸로 바꿔서 입력)

```
document.querySelectorAll('.review_text').forEach(el =>
  console.log(el.innerText))
```

1. 콘솔 창에 그 페이지에 있는 리뷰 텍스트가 **전부 한 번에** 쭉 출력됨
2. 리뷰가 더 필요하면: 페이지를 아래로 스크롤해서 리뷰를 더 불러온 다음(또는 "더보기" 버튼 클릭), 같은 명령어를 콘솔에 **다시 한 번** 입력 → 새로 로드된 리뷰까지 한꺼번에 또 뽑을 수 있음
3. 출력된 내용을 드래그해서 전체 복사

class 이름이 사이트마다 다르고, 사이트가 업데이트되면 바뀔 수도 있음. 명령어 쳤는데 아무것도 안 나오면 4-1로 돌아가서 class 이름을 다시 확인.

4-3. 뽑은 데이터 저장하기

헛갈리기 쉬운 부분이라 먼저 정리하면: 4-2에서 콘솔로 한 번에 뽑은 텍스트를, **파일 하나에 통째로 붙여넣기만 하면 끝**. 리뷰를 하나씩 복붙하거나 스크린샷 찍을 필요 전혀 없음. HTML 코드나 원본 화면을 따로 보관할 필요도 없음 — 4-1, 4-2는 텍스트를 뽑아내기 위한 과정일 뿐, 그 과정에서 본 HTML 코드 자체는 저장 안 해도 됨.

1. 4-2에서 콘솔에 출력된 리뷰 텍스트 전체를 한 번에 드래그해서 복사 → **메모장**에 붙여넣기 → review_raw.txt로 저장. 이게 전부임 (review_raw.txt 파일 하나에 여러 스타디카페 리뷰가 다 같이 들어가도 됨). 출처는 리뷰 하나하나마다 적을 필요 없고, 파일 맨 위에 "OO스타디카페 - 카카오톡 리뷰" 식으로 한 줄만 적어두면 충분함.
2. 최소 80~100문장 정도 모으는 걸 목표로 (데이터가 적으면 나중에 답변이 단조로워짐)

실습용으로만 쓰고, 크롤링한 데이터를 외부에 공개 게시하거나 상업적으로 쓰지 않기. 개인정보(작성자 닉네임 등)는 빼고 리뷰 본문만 저장.

4-4. (자주 묻는 질문) 표로 옮겨도 되나?

코드 상태로 뽑은 리뷰를 바로 엑셀 표로 정리해도 됨. 오히려 표로 관리하는 게 다음 단계(라벨링, Q&A 정리)에서 훨씬 편함. 메모장에는 원본만 백업해두고, 실제 작업은 표에서 하는 걸 추천.

4-5. 클래스(카테고리) 정하기

리뷰를 그냥 표로 옮기기만 하면 안 되고, 리뷰마다 **클래스(카테고리)**(리뷰 내용이 어떤 주제인지)랑 **톤**(긍정/부정)을 같이 적어야 챗봇이 질문 유형별로 답을 찾을 수 있음. 이 표를 만드는 과정 자체가 "라벨링"임.

클래스는 먼저 내가 직접 정하는 것부터 시작

아래 예시(혼잡도/소음/시설/몰입환경/운영시간)는 스터디카페 기준으로 잡아본 것일 뿐, 다른 업종이나 다른 기준으로 프로젝트를 짤다면 얼마든지 바뀌도 됨. 클래스 정하는 순서:

1. 크롤링한 리뷰를 30~40개 정도 훑어보면서, 자주 반복되는 주제 단어를 체크 (예: "조용하다/시끄럽다"가 자주 나오면 → 소음 클래스 후보)
2. 방문 전 사람들이 제일 궁금해할 만한 포인트 기준으로 추려보기 (내가 이 서비스를 왜 만들었는지랑 연결되는 기준이면 더 좋음)
3. **클래스 개수는 최대 4개까지로 잡는 걸 추천.** 2개 이하면 답변이 너무 뭉뚱그려지고, 4개를 넘어가면 클래스별로 리뷰가 쪼개져서 각 클래스에 답할 만한 리뷰 수가 부족해짐 (이 유인물 예시는 5개까지 늘린 케이스인데, 처음 시작할 땐 4개 이내로 잡고 나중에 필요하면 늘리는 걸 추천)
4. 클래스끼리 겹치는 게 있으면(예: "시설"이랑 "몰입환경"이 헷갈림) 처음부터 완벽하게 나누려 하지 말고, 일단 정해서 진행하고 라벨링하다가 겹치는 사례가 자주 나오면 그때 기준을 다듬으면 됨

클래스를 정했으면(이 유인물에서는 혼잡도/소음/시설/몰입환경/운영시간 예시로 계속 씀), 다음 단계에서 표 초안을 만든다.

참고: 카테고리를 4개로 시작했다가 나중에 부족한 걸 발견해서 늘리는 흐름은 5-4에서 실제 사례로 다룸.

4-6. 문장 단위로 쪼개기

한 리뷰 안에 여러 내용이 섞여 있는 경우가 많음. 이걸 그대로 통째로 라벨링하면 카테고리 하나로 뭉뚱그려져서 정보가 뭉개짐.

- 예시: "조용하고 좋은데 에어컨이 좀 세요" → 이 한 문장 안에 소음 얘기랑 시설 얘기가 같이 있음. 이걸 통째로 "소음"이나 "시설" 하나로 라벨링해버리면 나머지 정보가 사라짐
- 그래서 먼저 문장 단위로 쪼개야 함: "조용하고 좋아요" (소음/긍정) / "에어컨이 좀 세요" (시설/부정) 이렇게 2문장으로 분리
- 100문장 넘게 손으로 쪼개기는 힘드니까, 다음 4-7 라벨링 프롬프트에 "여러 내용이 섞인 문장은 먼저 문장 단위로 나눈 다음 분류해줘"라는 조건을 넣어서 한 번에 처리 (그래서 이 단계는 별도 프롬프트 없이, 4-7 프롬프트에 조건만 추가하는 식으로 진행)
- 쪼갬 결과가 만족스러우면 review_raw.txt도 쪼갬 문장 기준으로 업데이트해두기 (다음 Step에서 이 파일을 계속 재사용하니까)

4-7. AI 톨로 라벨링 표 초안 만들기

100문장 넘게 손으로 하나하나 카테고리/톤을 정하기는 힘드니까, AI 톨로 표 초안부터 만들고 시작하는 걸 추천:

- **Gemini** 사용 시: 구글시트랑 바로 연동되니까 편함. 리뷰 원문을 붙여넣고 "여러 내용이 섞인 문장은 먼저 문장 단위로 나눈 다음, 각 문장을 카테고리(혼잡도/소음/시설/몰입환경/운영시간)랑 톤(긍정/부정)으로 분류해서 표로 정리해줘, 구글시트로 만들어줘"라고 요청하면 시트 형태로 바로 나옴. (카테고리 이름은 4-5에서 내가 정한 걸로 바꿔서 입력)
- **ChatGPT (유료 Plus)** 사용 시: 답변을 **엑셀 파일(.xlsx)**로 바로 내보내는 기능이 있음. "문장 단위로 나누고 표로 정리해서 엑셀 파일로 다운로드하게 해줘"라고 요청하면 파일로 바로 받을 수 있음. (무료 버전은 파일 다운로드가 안 되니까, 표만 받아서 직접 엑셀에 옮겨 붙이면 됨)

프롬프트 작성 팁: 위 두 요청 문장에는 공통으로 이런 게 들어가 있음 - **문장 분리 조건을 먼저 넣기** — "여러 내용이 섞인 문장은 먼저 나눠줘"를 안 넣으면 한 리뷰가 통째로 한 카테고리라벨링돼버림 - **분류 기준(카테고리/톤)을 요청 문장 안에 직접 나열하기** — "분류해줘"라고만 하면 AI가 임의로 기준을 만들어버림. 내가 정한 클래스 이름을 그대로 문장에 넣어야 함 - **출력 형태를 못박기** — "표로", "구글시트로", "엑셀 파일로"처럼 결과물 형태를 구체적으로 지정해야 나중에 옮기는 수고가 줄어듦

여기서 만든 표는 **초안**임. 한 모델한테만 시킨 라벨링이라 정확하지 않을 수 있어서, 다음 Step(3모델 교차 검증)에서 이 표를 기준으로 GPT/Claude/Gemini 세 개 다 시키고 다수결로 최종 확정할 거임. 교차검증

끝나고 라벨을 한 방향으로 굳힐 때는 예외 없이 **다수결 기준**으로만 정한다 (내 개인 판단으로 임의로 바꾸지 않기).

5. Step 3 — LLM 교차검증으로 리뷰 라벨링 확정하기

4-7에서 만든 표는 한 모델 기준 초안일 뿐. 이걸 그대로 쓰면 그 모델 기준이 곧 정답이 되는 셈이라, GPT/Claude/Gemini 세 모델에 각각 다시 시키고 다수결로 최종 라벨을 정한다.

5-1. 세 모델에 동일한 프롬프트로 라벨링 요청

ChatGPT, Claude, Gemini 각각에 **똑같은 프롬프트**로 리뷰 문장들을 넣기 (조건이 달라지면 비교가 안 되니까 프롬프트는 통일).

프롬프트 예시

아래 리뷰 문장들을 하나씩 분류해줘.

카테고리 (다음 중 하나만 선택): 혼잡도 / 소음 / 시설 / 몰입환경 / 운영시간
톤 (다음 중 하나만 선택): 긍정 / 부정 / 중립

표 형식으로 정리해줘: 리뷰 원문 | 카테고리 | 톤

리뷰 문장:

[review_raw.txt 내용 붙여넣기]

프롬프트 작성 팁: 이 프롬프트에는 3가지가 꼭 들어가야 함 - **선택지를 명확히 제한하기** — "카테고리 중 하나만", "톤 중 하나만"처럼 고를 수 있는 값을 딱 정해줘야 함. 안 그러면 모델마다 "약간 긍정적" 같은 애매한 답을 만들어내서 세 모델 결과 비교가 안 됨 - **출력 형식 지정하기** — "표 형식으로", 컬럼 순서까지 적어줘야 나중에 표로 옮기기 편함 - **원본 그대로 넣기** — 리뷰 문장은 요약하지 말고 원문 그대로 붙여넣기 (요약하면 뉘앙스가 사라져서 톤 판단이 부정확해짐)

5-2. 세 결과를 표로 나란히 정리

구글 스프레드시트에 아래처럼 정리:

리뷰 원문	GPT 라벨	Claude 라벨	Gemini 라벨	최종(다수결)
"시험기간엔 자리 없을 정도로 조용해요"	소음/긍정	소음/긍정	소음/긍정	소음/긍정
"인테리어 아늑해서 좋아요"	시설/긍정	몰입환경/긍정	시설/긍정	시설/긍정 (2표)

5-3. 다수결로 최종 확정

- **다수결 규칙:** 3개 중 2개 이상 일치하면 그걸로 확정
- 3개가 다 다르게 나오면 직접 판단해서 정하고, 왜 그렇게 정했는지 한 줄 메모해두기 (나중에 발표할 때 근거 자료로 씀)
- 일치율도 계산해두면 좋음 (전체 문장 중 몇 %가 3모델 만장일치였는지) — "그냥 내가 정한 게 아니라 교차검증했다"는 근거가 됨

5-4. 카테고리별 데이터 개수 확인하고 부족하면 보강하기

라벨링이 다 끝났으면 카테고리별로 몇 문장씩 있는지 세어보기. 특정 카테고리가 너무 적으면(예: 10개 미만) 그 카테고리로는 챗봇이 제대로 된 답을 못 만들.

1. 엑셀에서 =COUNTIF(카테고리열, "혼잡도")처럼 카테고리별 개수 세기
2. 너무 적은 카테고리가 있으면 4-2(콘솔 크롤링)로 돌아가서 해당 카테고리 관련 리뷰를 더 모으거나, 다른 스터디카페 리뷰까지 추가로 크롤링해서 보강
3. 보강한 리뷰도 똑같이 4-6(문장 쪼개기) → 4-7(라벨링 초안) → 5-1~5-3(교차검증) 과정을 다시 거쳐서 표에 추가

처음부터 카테고리를 다 완벽하게 채우려고 하지 않아도 됨. 실제로 이 프로젝트도 처음엔 혼잡도/시설/소음/몰입환경 4개로 시작했는데, 나중에 프로토타입을 테스트해보니 "몇 시까지 해?" 같은 질문에 답을 못 하는 게 발견돼서 운영시간 카테고리를 추가로 만들고 리뷰를 새로 모았음. 그래서 4-5에서 "클래스는 최대 4개로 시작하고 나중에 필요하면 늘려도 된다"고 한 거임 — 실제로 이렇게 진행됐던 사례.

6. Step 4 — 질문-답변 데이터셋 구성하기

라벨링 끝난 리뷰를 카테고리별로 묶어서, 실제 챗봇이 답할 질문-답변 쌍으로 만든다. 최소 15~20개. 이것도 4-7처럼 손으로 다 만들지 않고 AI 시켜서 표로 뽑는다.

6-1. 카테고리별로 리뷰 모으고 개수 세기

1. Step 5(교차검증)에서 최종 확인한 라벨링 표(리뷰 원문 | 카테고리 | 톤)를 엑셀/구글시트에서 **카테고리 기준으로 정렬 또는 필터**해서, 카테고리별로 리뷰가 몇 개씩 있는지 눈으로 먼저 확인
2. 카테고리별 긍정/부정 개수는 엑셀 함수로 셀 수 있음: =COUNTIFS(카테고리열, "소음", 톤열, "긍정") 식으로. 함수 쓰기 귀찮으면 이것도 그냥 AI한테 시켜도 됨 (6-2에서 한 번에 처리)

6-2. AI한테 질문-답변 표 만들어달라고 요청

Step 5에서 만든 최종 라벨링 표 전체를 ChatGPT나 Gemini에 붙여넣고 아래처럼 요청.

프롬프트 예시

아래는 카테고리(혼잡도/소음/시설/물입환경/운영시간)와 톤(긍정/부정)으로 라벨링이 끝난 스터디카페 리뷰 데이터야.

[Step 5 최종 라벨링 표 붙여넣기]

이 데이터로 리뷰 기반 챗봇용 질문-답변 데이터셋을 표로 만들어줘. 각 카테고리마다 사람들이 실제로 물어볼 만한 자연스러운 질문을 2~4개씩 만들고, 아래 규칙에 따라 표를 완성해줘.

표 컬럼: 질문 유형 | 키워드 | 카테고리 | 긍정 리뷰 수 | 부정 리뷰 수 | 답변 상태 | 답변 문구 | 원문 리뷰 예시

규칙:

- 긍정 리뷰 수, 부정 리뷰 수는 해당 카테고리의 실제 라벨링 데이터에서 세어줘
- 긍정:부정 비율이 7:3 밖이면 "확실 답변", 7:3 안쪽이면 "의견 갈림"으로 답변 상태를 판정해줘
- 관련 리뷰가 3개 미만인 카테고리는 답변 상태를 "정보부족"으로 처리해줘
- 답변 문구는 "대체로 ~하다는 평이 많아요 (9/10)"처럼 리뷰 수 근거를 넣어서 자연스럽게 써줘
- 원문 리뷰 예시는 실제 라벨링 데이터에 있는 문장을 그대로 가져와줘 (지어내지 말 것)

프롬프트 작성 팁: 이 프롬프트에는 4가지가 꼭 들어가야 함 - **원본 데이터를 통째로 붙여넣기** — 요약본이나 일부만 주면 AI가 개수를 잘못 셈. 라벨링 표 전체를 넣어야 정확한 통계가 나옴 - **출력 컬럼을 순서대로 명시하기** — 컬럼명과 순서를 정확히 적어줘야 표로 바로 옮겨 붙일 수 있는 형태로 나옴 - **판정 기준을** 숫

자로 못박기 — "적당히 많으면", "충분하면" 같은 표현 대신 "7:3", "3개 미만"처럼 정확한 숫자를 줘야 매번 같은 기준으로 판정됨 - **지어내지 말라고 명시하기** — 원문 리뷰 예시처럼 실제 데이터에서 가져와야 하는 항목은 "지어내지 말 것"을 꼭 붙이기. 안 그러면 그럴듯한 문장을 새로 만들어버릴 때가 있음

6-3. 부족하면 더 요청하고 저장하기

- 최소 15~20개가 나올 때까지 카테고리별로 질문을 더 요청해도 됨 ("소음 카테고리 질문 2개만 더 만들어줘" 식으로)
- 결과는 구글시트/엑셀로 옮겨서 표로 보관 (Gemini는 구글시트 바로 연동, ChatGPT Plus는 엑셀 다운로드 — 4-7에서 쓴 방식과 동일)

6-4. 결과 검토하기

AI가 만든 답변 상태(확실/의견갈림/정보부족)와 비율이 실제 라벨링 데이터랑 맞는지 몇 개는 직접 대조해서 확인. AI가 개수를 잘못 세는 경우가 가끔 있어서, 전부 그대로 믿지 말고 표본으로 검산 한 번 해두는 걸 추천.

완성 표 예시

질문 유형	키워드	카테고리	긍정 리뷰 수	부정 리뷰 수	답변 상태	답변 문구	원문 리뷰 예시
시험기간에도 조용해?	시험기간, 조용, 소음	소음	9	1	확실 답변	"대체로 조용하다는 평이 많아요 (9/10)"	"시험기간엔 자리 없을 정도로 조용해요"
시설 깨끗해?	시설, 깨끗, 청결	시설	5	5	의견 갈림	"의견이 갈려요. 최근 리뷰 확인 추천"	"화장실이 좀 아쉬웠어요"

- 긍정:부정 비율이 **7:3** 밖이면 **확실 답변**, **7:3** 안쪽이면 **의견 갈림**
 - 관련 리뷰가 3개 미만이면 무조건 "정보부족" 상태로
-

7. Step 5 — Framer AI로 화면 시안 만들기

기능 연결은 안 하고, 디자인 참고용 시안만 만드는 단계.

7-1. AI 프롬프트로 화면 만들기

Framer 가입 → 새 프로젝트 → AI 프롬프트 입력창에 아래처럼 씀

스터디카페 리뷰 기반 Q&A 챗봇 모바일 앱 UI를 만들어줘.
- 다크 배경(#17171C) + 민트 포인트 컬러(#3FA57A)
- 상단: 챗봇 대화창 (사용자 말풍선 오른쪽, AI 말풍선 왼쪽)
- 하단: 질문 입력창 + 추천 질문 칩 3개
- AI 답변 카드에는 "확실 답변" 또는 "의견 같림" 배지, 긍정/부정 비율 바, "원문 리뷰 보기" 버튼 포함
- 375x812 모바일 사이즈

프롬프트 작성 팁: 이 프롬프트에는 3가지가 꼭 들어가야 함 - **색상 코드를 정확한 값으로 주기** — "어두운 배경에 민트색" 처럼 두루뭉술하게 쓰면 매번 다른 색이 나옴. #17171C, #3FA57A처럼 정확한 헥스코드로 지정 - **화면 안의 요소를 위치까지 구체적으로 나열하기** — "상단에 뭐가 있고 하단에 뭐가 있는지"까지 적어야 레이아웃이 흔들리지 않음 - **사이즈를 명시하기** — 모바일/데스크탑 여부, 픽셀 사이즈를 안 적으면 임의 비율로 나와서 나중에 화면끼리 사이즈가 안 맞을 수 있음

7-2. 필요한 화면 6개 만들고 저장하기

- 필요한 화면 6개(위 상태표 기준: 대기/로딩/확실답변/의견같림/정보부족/이해실패) 만들고 각각 스크린샷으로 저장해두기
- 한 번에 다 요청하지 말고 "로딩 화면만 다시 만들어줘" 식으로 화면별로 하나씩 다듬으면 결과가 더 깔끔함

7-3. 화면 안의 요소마다 영문 이름 붙이기

Framer에서 화면을 만들면 버튼, 아이콘, 텍스트 같은 요소들이 각각 레이어로 나뉘어 있음. 이 레이어들 이름을 기본값(Frame 1, Rectangle 2 같은 거) 그대로 두지 말고, 뭘 하는 요소인지 알 수 있는 영문 이름으로 하나하나 바꿔주기.

1. Framer 왼쪽 **Layers** 탭에서 이름 바꿀 레이어 더블클릭 → 새 이름 입력

2. 예시: 질문 입력창 → `QuestionInput`, 전송 버튼 → `SendButton`, 확실답변 배지 → `AnswerBadge`, 원문 보기 버튼 → `ToggleReviewButton`, 비율 바 → `RatioBar`
3. 규칙: 버튼은 `~Button`, 표시용 텍스트/배지는 `~Badge`나 `~Label`, 입력창은 `~Input` 처럼 끝에 역할을 붙이면 나중에 봐도 헷갈리지 않음

왜 이렇게 해야 하는지: 8-1 단계에서 ChatGPT한테 "이 이름 그대로 코드에 써줘"라고 시키면, 화면 요소랑 코드 안 변수/id 이름이 정확히 일치하게 됨. 그래야 8-4에서 "SendButton 눌렀을 때 QuestionInput 값을 가져와서 처리해줘" 처럼 요소 이름으로 정확하게 기능을 지정할 수 있음. 이름을 안 붙이고 넘어가면 ChatGPT가 임의로 이름을 지어버려서, 원하는 요소에 원하는 기능을 정확히 붙이기가 훨씬 어려워짐.

7-4. 요소 이름표 정리하기

다 붙였으면 요소 이름 목록을 표로 따로 정리해두기 (다음 단계에서 프롬프트에 그대로 붙여넣을 거임)

요소	영문 이름
질문 입력창	<code>QuestionInput</code>
전송 버튼	<code>SendButton</code>
확실답변 배지	<code>AnswerBadge</code>
원문 보기 버튼	<code>ToggleReviewButton</code>
긍정/부정 비율 바	<code>RatioBar</code>

이 시안(스크린샷 + 요소 이름표)은 8번 단계에서 ChatGPT한테 코드 짜달라고 할 때 참고 자료로 씬.

8. Step 6 — ChatGPT로 완성 코드 받기

이 단계는 네 번에 나눠서 진행. 한 번에 "디자인 + 기능"을 다 시키면 결과가 뒤죽박죽되기 쉬우니까, **화면부터 코드로 만들고, 그다음에 기능을 붙이는 순서로** 간다.

8-1. 디자인 시안을 코드로 변환 요청하기

아직 기능은 넣지 않고, 화면 생김새만 코드로 옮기는 단계. Step 5에서 만든 Framer 화면 6개 스크린샷 + 요소 이름표를 준비해서 ChatGPT에 이미지들을 첨부하고 아래처럼 요청.

첨부한 이미지들은 스터디카페 리뷰 Q&A 챗봇 화면 시안이야 (대기/로딩/확실답변/의견달림/정보부족/이해실패 순서).

이 화면들을 HTML + CSS로 최대한 똑같이 만들어줘.

조건:

- 아직 기능(버튼 클릭, 데이터 처리)은 넣지 마. 화면 구조와 스타일만 완성해줘.
- 색상은 다크 배경(#17171C) + 민트 포인트(#3FA57A) 그대로 써줘.
- 화면 6개를 각각 다른 div 섹션으로 나눠서 한 파일 안에 넣어줘. (전환 처리는 다음 단계에서 할 거임)
- 모바일 사이즈(375x812) 기준으로 만들어줘.
- 버튼, 입력창, 배지 같은 요소들은 아래 이름표에 있는 이름을 그대로 id로 써줘. (예: 전송 버튼 → id="SendButton")

[Step 5에서 만든 요소 이름표 붙여넣기]

프롬프트 작성 팁: 이 프롬프트에는 5가지가 꼭 들어가야 함 - **화면 순서와 개수를 먼저 알려주기** — 이미지가 몇 장인지, 어떤 순서인지 말해줘야 AI가 각 이미지를 헷갈리지 않고 매칭함 - **"기능은 넣지 마"를 명시하기** — 안 그러면 AI가 알아서 버튼에 동작을 추측해서 넣어버려서, 다음 단계에서 오히려 코드가 꼬임 - **색상 값을 다시 한번 정확히 주기** — 이미지에서 색이 조금 다르게 나올 수 있어서, 헥스코드를 프롬프트에 또 적어주는 게 안전함 - **화면을 어떻게 쪼갤지 말해주기** — "화면 6개를 각각 다른 div 섹션으로 나눠줘"라고 요청하면, HTML 안에 화면마다 <div id="대기화면">, <div id="로딩화면"> 이런 식으로 6개 상자가 따로따로 만들어짐. 이렇게 미리 나눠놔야 나중에 "지금은 로딩화면 상자만 보여주고 나머지는 숨겨줘" 같은 코드를 붙이기 쉬움. 만약 이 요청을 안 하면 화면 6개가 구분 없이 한 덩어리로 뒤섞여 나올 수 있고, 그러면 상태별로 화면 전환시키는 코드를 짜기가 훨씬 번거로워짐 - **Step 5에서 만든 요소 이름표를 그대로 붙여넣기** — Framer에서 붙인 영문 이름(SendButton, QuestionInput 등)을 코드의 id로 그대로 쓰라고 시켜야, 8-4에서 "SendButton 눌렀을 때"처럼 요소를 정확히 콕 집어서 기능을 지정할 수 있음. 이걸 안 하면 ChatGPT가 btn1, submit-btn 같은 임의의 이름을 붙여버려서 요소를 찾기가 헷갈려짐

8-2. 화면별로 다듬기

- 결과 보면서 "로딩 화면만 다시 만들어줘" 식으로 화면별로 하나씩 다듬기
- 여기까지 만들어진 코드는 **정적인 화면**. 버튼을 눌러도 아직 아무 반응이 없는 게 정상.

8-3. 엑셀/구글시트 표를 프롬프트에 넣는 방법

Step 4에서 만든 데이터가 엑셀이나 구글시트에 표로 정리돼 있을 텐데, 이걸 텍스트로 옮기는 건 어렵지 않음. 방법 두 가지:

- **복사-붙여넣기 (제일 간단, 추천):** 엑셀/구글시트에서 표 범위(제목 줄 포함)를 마우스로 드래그해서 선택 → 복사(Ctrl+C) → ChatGPT 채팅창에 그대로 붙여넣기(Ctrl+V). 셀 단위로 복사한 표는 자동으로 줄바꿈+탭 구분된 텍스트로 붙여지는데, ChatGPT가 이 형태를 표로 잘 인식함. 파일 변환 같은 거 따로 안 해도 됨.
- **파일 자체를 첨부 (ChatGPT Plus만 가능):** 유료 버전이면 채팅창 옆 첨부 버튼으로 .xlsx 파일을 그대로 올릴 수 있음. 이 경우 프롬프트에서 "[표 붙여넣기]" 자리에 텍스트 대신 "첨부한 엑셀 파일이 질문-답변 데이터야"라고 문장으로 바꿔주면 됨. 무료 버전은 파일 첨부이 안 되니까 복사-붙여넣기로 진행.

Step 6-2, Step 5-1처럼 다른 단계에서 "표 붙여넣기"라고 되어 있는 부분도 전부 같은 방식(복사-붙여넣기 또는 파일 첨부)으로 하면 됨.

8-4. 실제 작동하는 기능 코드 요청하기

8-1에서 받은 HTML/CSS 코드 + Step 4에서 만든 질문-답변 데이터 표를 같이 붙여넣고 요청.

아래는 지금까지 만든 화면 코드야.

[8-1에서 받은 HTML/CSS 붙여넣기]

그리고 아래는 라벨링 완료된 스터디카페 리뷰 Q&A 데이터야.

[Step 4 표 붙여넣기]

이 화면에 실제로 작동하는 기능을 추가해줘. 조건은 다음과 같아.

- 자바스크립트로 질문 입력 → 매칭 → 화면 상태 전환이 되게 만들어줘.
- 실제 API 호출 없이, 위 데이터 안에서만 키워드 매칭으로 답을 찾아야 해.
- 화면 상태는 대기/로딩/확실 답변/의견 같림 답변/정보부족/이해실패 6개로 관리해줘. (8-1에서 나눠둔 div들을 상태에 따라 보이고 숨기는 식으로)
- 의견 같림 기준은 긍정:부정 비율 7:3.
- html, css, javascript를 전부 파일 하나(index.html) 안에 합쳐줘. 외부 파일 연결 없이 더 클릭하면 바로 실행되는 형태로.
- 기능은 요소 id 이름(SendButton, QuestionInput 등) 기준으로 정확히 연결해줘. 예를 들어

"SendButton을 클릭하면 QuestionInput의 값을 가져와서 매칭 로직을 실행"하는 식으로.

- 주석은 한글로, 초보자도 읽을 수 있게 달아줘.

프롬프트 작성 팁: 이 프롬프트에는 6가지가 꼭 들어가야 함 - **이전 단계 결과물을 그대로 붙여넣기** — 8-1 코드를 새로 설명하지 말고 그대로 붙여넣어야 화면 구조가 안 바뀜 - **"실제 API 호출 없이"**를 명시하기 — 안 적으면 AI가 임의로 fetch 코드나 API 연동 흉내를 넣어버릴 수 있음 - **상태 이름을 정확히 나열하기** — 6개 상태 이름을 그대로 적어야 코드 안 변수명이랑 8-1의 div 이름이 맞아떨어짐 - **파일을 합쳐달라고 명시하기** — "index.html 하나로, 외부 파일 연결 없이"라고 안 하면 css/js를 별도 파일로 분리해서 줄 때가 있어서 메모장 저장 단계에서 곤란해짐 - **요소 id 이름으로 콕 집어서 지시하기** — "버튼 누르면"처럼 두루뭉술하게 말하지 말고, Step 5에서 붙인 이름을 그대로 써서 "SendButton을 클릭하면"처럼 구체적으로 지시. 그래야 원하는 요소에 원하는 기능이 정확히 연결됨 - **주석 요청하기** — 코드를 이해하고 넘어가려면 한글 주석을 꼭 요청

코드가 길면 "이어서 계속 작성해줘"로 이어받기.

9. Step 7 — 메모장에 저장해서 브라우저로 열기

9-1. 코드 복사하기

ChatGPT가 준 코드 전체를 복사 (``html 코드블록 안의 내용 전부)

9-2. 메모장에 저장하기

1. **메모장** 실행 (윈도우 시작 메뉴에서 "메모장" 검색)
2. 복사한 코드를 메모장에 붙여넣기
3. 메모장 상단 **파일 → 다른 이름으로 저장**
4. 저장 창에서
5. 파일 이름: `review_chatbot.html` (반드시 뒤에 `.html`까지 직접 입력)
6. 파일 형식: **모든 파일**로 변경 (그냥 두면 자동으로 `.txt`가 붙어버림)

9-3. 브라우저로 열기

1. 저장한 파일 찾아서 **더블클릭**

- 2. 메모장으로 열리면: 파일 우클릭 → 연결 프로그램 → **Chrome** 선택해서 다시 열기
- 3. 크롬에서 챗봇 화면이 뜨면 성공

10. Step 8 — 테스트하기

Step 4 표에 있는 질문들을 실제로 입력해보면서 상태 6개가 다 잘 뜨는지 하나씩 확인.

- 확실히 답변 케이스
- 의견 갈림 케이스
- 정보부족 케이스
- 아예 관련 없는 질문 넣어서 이해실패 케이스까지 전부 테스트

11. 자주 나는 문제

증상	원인	해결
한글이 깨져서 나옴	저장 시 인코딩 문제	메모장에서 다시 저장할 때 인코딩을 UTF-8로 지정
더블클릭해도 메모장으로만 열림	파일이 .html.txt로 저장됨	파일 형식을 "모든 파일"로 바꿔서 다시 저장
버튼 눌러도 반응 없음	자바스크립트 에러	F12 → Console 탭에서 빨간 에러 메시지 확인 → 에러 내용 그대로 ChatGPT한테 붙여넣고 "이 에러 고쳐줘"
항상 "이해실패"만 뜸	키워드 매칭 조건이 너무 좁음	ChatGPT에 "키워드 매칭을 조금 더 느슨하게 해줘"로 재요청
크롤링한 텍스트에 광고/댓글까지 같이 뺄힘	class 선택이 리뷰 영역 말고 다른 요소까지 포함	4-1로 돌아가서 더 구체적인 class 다시 확인

12. 마무리 체크리스트

- [] 리뷰 원본 80~100문장 이상 크롤링했다
- [] 여러 내용이 섞인 문장은 문장 단위로 쪼갰다
- [] 표로 정리하고 카테고리+톤 초안 라벨링까지 했다
- [] GPT/Claude/Gemini 3모델 교차검증 + 다수결로 최종 라벨 확정했다
- [] 카테고리별 데이터 개수를 확인하고, 부족한 카테고리는 보강했다
- [] 질문-답변 데이터 15개 이상 정리했다
- [] Framer로 화면 시안 6개 만들었다
- [] 화면(HTML/CSS)과 기능(JS)을 나눠서 각각 받았다
- [] 메모장 → .html 저장 → 크롬 실행까지 확인했다
- [] 상태 6개 다 테스트해봤다

여기까지 되면 배포 없이도 프로토타입 완성. 데모 발표할 땐 이 html 파일 하나만 노트북에 들고 가서 크롬으로 열면 됨.